

DukaSmart

A Web-Based Demand Forecasting and Budget-Constrained Inventory Decision Support System for Small Retail Shops in Nairobi County, Kenya

Richard Musili Mwendwa · SCT222-0320/2022

Supervisor: **Dennis Njagi**

BSc Business Computing — JKUAT — 2026

The Problem

Small shop owners must decide every week:

- **What** to restock?
- **How much** to buy?
- **What to buy first** when money is limited?

But their records only show what happened — **not what to do next.**

- **8%** of items customers want are out of stock worldwide (Gruen et al., 2002)
- **7.4 million MSMEs** in Kenya (KNBS, 2016)
- Low stock alone doesn't tell the owner which product deserves the next shilling

Research Objectives

General objective: Design, develop and evaluate an explainable web-based demand forecasting and budget-constrained inventory decision support system for small retail shops in Nairobi County.

Specific objectives:

1. Investigate current sales recording, stock monitoring and purchasing practices
2. Determine an appropriate, interpretable short-term forecasting approach
3. Design and implement an explainable budget-constrained replenishment model
4. Evaluate forecast accuracy, correctness, usability and consistency

Literature Gap

What the literature already gives us:

- Forecasting methods that work for large retailers (Fildes et al., 2022; Makridakis et al., 2022)
- Model selection per series (Ma & Fildes, 2021; Ulrich et al., 2022)
- Integrated forecasting and inventory control (van der Haar et al., 2024)

The four gaps this project fills:

1. Small shops have short, noisy data — big-chain methods don't transfer
2. Forecasting studies stop at the forecast, not a budget-ranked plan
3. Kenyan literature describes practice but provides no tool
4. Existing tools give answers without reasons

Proposed Solution

Workflow

Sales history → Demand forecast → Compare with stock → Calculate replenishment → Estimate gross profit → Check budget → **Prioritised purchase plan**

Key operations

- **Forecasting:** 3 models compete (WMA, SES, Gradient Boosting) — lowest MAE wins per product
- **Priority score:** $0.40 \times \text{stockout risk} + 0.30 \times \text{demand} + 0.20 \times \text{gross profit} + 0.10 \times \text{affordability}$
- **Budget allocation:** Greedy, top-down by priority rank, partial fills allowed
- **Explainability:** Every recommendation carries a plain-language reason
- **Control stays with the owner:** Accept, change or skip each line

Methodology

Approach: Design Science (Hevner et al., 2004; Peffers et al., 2007) + Agile iterative development

Forecast validation: 14-day time-ordered holdout, MAE, no shuffling

Sufficiency gate: Minimum 10 non-zero sales days AND 28-day history span

Technology stack

- Backend: Python 3.14, FastAPI, SQLAlchemy, SQLite
- ML: pandas, NumPy, scikit-learn, statsmodels
- Frontend: React 18, Vite 6, Tailwind 4, Recharts
- Auth: bcrypt + JWT HS256

Architecture

Layered client-server design

- **Presentation:** React single-page app
 - Login · Home · Record sale · Stock · What to buy · Sales outlook · Products · Suppliers · Reports
- **Application:** FastAPI routes + bearer token check
- **Domain:** inventory.py · forecasting.py · recommendation.py · config.py
- **Data:** SQLAlchemy models + SQLite

See Figure 3.13 (Component Diagram) in the report.

Key Code

Stock is derived, never stored

```
def current_stock(db, product_id):  
    total = db.execute(  
        select(func.coalesce(func.sum(InventoryMovement.change_qty), 0))  
        .where(InventoryMovement.product_id == product_id)  
    ).scalar_one()  
    return int(total or 0)
```